

Programmieren lernen mit Python IDLE



Arbeitsblätter zum Kurs

Inhalt

Programmzeilen mit der IDLE Shell testen	2
Mit der IDLE Shell rechnen	3
Mit dem IDLE Editor ein Programm schreiben	4
Aufgabe: Ziffern mit Punkten darstellen	5
Eine Liste speichert viele änderbare Elemente	6
Ein Tupel speichert viele nicht änderbare Elemente	7
Ein Dictionary speichert Paare von Schlüssel und Wert	8
Der Computer fragt	9
Der Computer unterscheidet zwischen "wahr" und "falsch"	10
Der Computer unterscheidet Fälle	11
Der Computer dreht Schleifen	12
Aufgabe: Dialog mit dem Benutzer	13
Funktionen haben Input und Output: Ganzzahlen	14
Funktionen haben Input und Output: Tupel, Liste	15
Globale und lokale Variable in der Funktion	16
Aufgaben mit Funktionen lösen: Konstruktionsanleitung	17
Aufgaben mit Funktionen lösen: Tabelle drucken	18
Aufgaben mit Funktionen lösen: Diagramm plotten	19
Klassen haben Eigenschaften und Methoden	20
Die Eltern-Klasse vererbt – die Kind-Klasse erbt	21
Roboter im Irrgarten: Wände bauen	22
Roboter im Irrgarten: Wände ablegen	23
Roboter im Irrgarten: Wände aus Datei lesen	24
Roboter im Irrgarten: Schildkröte bewegen	25
Roboter im Irrgarten: Schauen, gehen, drehen	26
Roboter im Irrgarten: Rechte-Hand-Methode	28
Roboter im Irrgarten: Pledge-Algorithmus	29
Auto im Gegenverkehr: Auto steuern	31
Auto im Gegenverkehr: Gegenverkehr kommt	33
Auto im Gegenverkehr: Zusammenstoß melden	34
Auto im Gegenverkehr: Gegenverkehr wird schneller	36
Auto im Gegenverkehr: Straße und Sound	37
Ein Dinosaurier überquert die Straße	38
Quellen	39

Programmzeilen mit der IDLE Shell testen

Gib die Programmzeilen nach dem Prompt (>>>) ein. Ist das Ergebnis OK oder bekommst du eine Fehlermeldung?

Tipp: Mit ALT + p kannst du die letzte Zeile wiederholen.

Zeichenketten (strings) testen

Programmzeile	OK	Fehler
Hallo Welt		
"Hallo Welt"		
"Hallo Welt'		
'Hallo Welt'		
print("Hallo Welt")		
pirnt("Hallo Welt")		

Zeichenketten in Variablen schreiben

Programmzeile	OK	Fehler
nachricht = Hallo Welt		
nachricht = "Hallo Welt"		
nachricht		
print(nachricht)		
len(nachricht)		
type(nachricht)		
nachricht.upper()		

Zahlen (integer und float) testen

Programmzeile	OK	Fehler
3		
3.14		
3,14		
print(3)		
print(3.14)		
print(3,14)		

Zahlen in Variablen schreiben

Programmzeile	OK	Fehler
zahl = 3		
zahl = 3.14		
zahl		
print(zahl)		
len(zahl)		
type(zahl)		
zahl.upper()		

Die Farbe kennzeichnet einen Text mit besonderer Bedeutung

Farbe	Bedeutung
rot	
grün	
violett	

Mit der IDLE Shell rechnen

Gib die Programmzeilen nach dem Prompt (>>>) ein. Ist das Ergebnis OK oder bekommst du eine Fehlermeldung?

Rechnen mit Zeichenketten

Programmzeile	OK	Fehler
"Hallo" + "Welt"		
"Hallo" + " " + "Welt"		
"Hallo " + "Welt"		
"Hallo " - "Welt"		
print("Hallo " + "Welt")		
3 * "Hallo Welt"		
3 * "Hallo Welt "		
3 / "Hallo Welt "		
print(3 * "Hallo Welt ")		
nachricht = "Hallo Welt "		
print(3 * nachricht)		

Rechnen mit Zahlen

Programmzeile	OK	Fehler
3 + 4		
3 - 4		
3 * 4		
3/4		
30//4		
30%4		
ergebnis = 3 * 4		
ergebnis/5		
ergebnis//5		
print(ergebnis/5)		

Punktrechnung vor Strichrechnung

Programmzeile	OK	Fehler
3 + 4 * 2		
(3 + 4) * 2		
12 - 3 / 2		
(12 - 3) / 2		
12 - 3 // 2		
(12 - 3) // 2		

Zahl in Zeichenkette umwandeln – Zeichenkette in Zahl umwandeln

Programmzeile	OK	Fehler
zahl = 3		
str(zahl)		
zahl = 3.14		
str(zahl)		
zeichenkette = "3.14"		
int(zeichenkette)		
float(zeichenkette)		

Mit dem IDLE Editor ein Programm schreiben

Mit dem Menüpunkt File/ New File erzeugst du ein leeres Editor-Fenster mit dem Titel "untitled".

Gib die Programmzeilen im Editor-Fenster ein. Die Farbe dort kennzeichnet einen Text mit besonderer Bedeutung.

Speichere den Inhalt mit dem Menüpunkt File/ Save unter dem Dateinamen "mein_erstes_programm" im Ordner Python/03_Rechnen_mit_Zeichenketten_und_Zahlen

Starte das Programm mit dem Menüpunkt Run/ Run Module.

Notiere die print-Ausgaben in der Tabelle.

Rechnen

Programmzeile	print-Ausgabe
# Das ist ein Kommentar	
# Rechnen	
nachricht = "Hallo Welt "	
print(3 * nachricht)	
ergebnis = 3 * 4	
print(ergebnis/5)	

Umwandeln

Programmzeile	print-Ausgabe
# Umwandeln	
zahl = 3.14	
print(zahl)	
print(str(zahl))	
zeichenkette = "3.14 "	
print(zeichenkette)	
print(3 * zeichenkette)	
print(3 * float(zeichenkette))	

Variablen ausgeben

Programmzeile	print-Ausgabe
# Variablen ausgeben	
print("nachricht =", nachricht)	
print("ergebnis =", ergebnis)	
print("zahl =", zahl)	
print("zeichenkette =", zeichenkette)	

Aufgabe: Ziffern mit Punkten darstellen

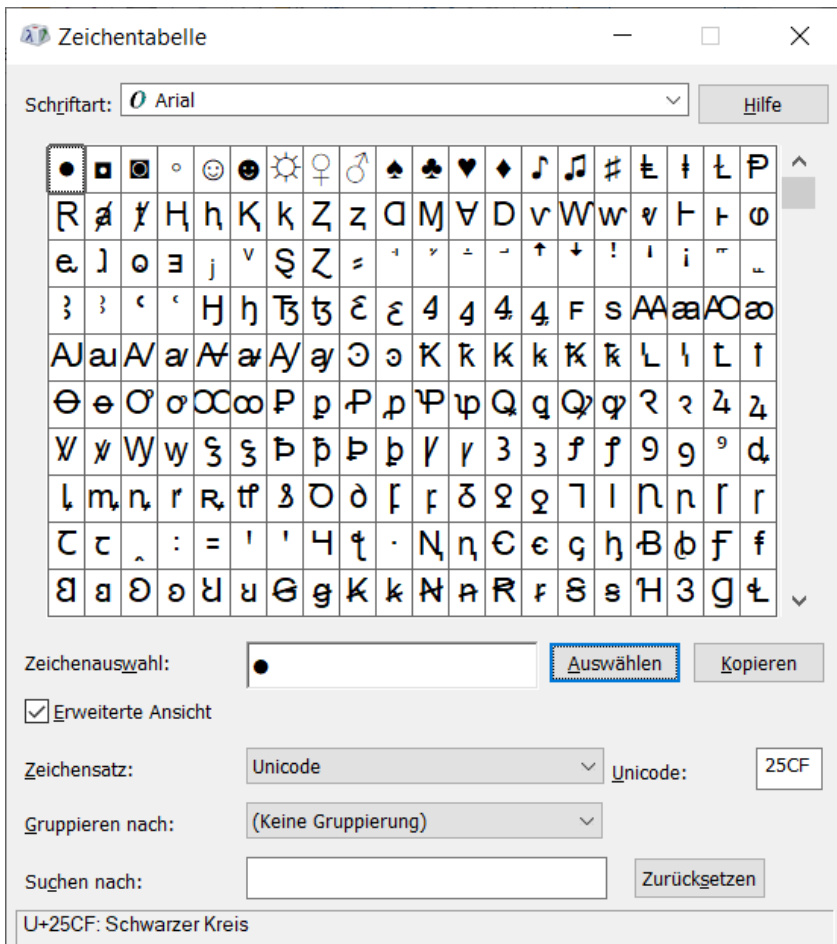
Die Ziffern auf einem Kassenzettel sind aus Punkten zusammengesetzt. Mit 5x7 Punkten können die Ziffern 0 bis 9 gut lesbar dargestellt werden.

	•	•	•	
•				•
•				•
•				•
•				•
•				•
	•	•	•	

Schreibe ein Programm, das mit print-Befehlen und dem Zeichen * oder • eine Ziffer in ein 5x7 Raster druckt:

- Drucke eine Null
- Drucke eine Acht
- Drucke eine Eins
- Drucke eine Ziffer deiner Wahl

Tipp: Das Zeichen • hat den Unicode 25CF. Es kann über die Windows-Zeichentabelle eingegeben werden.



Eine Liste speichert viele änderbare Elemente

Gib die Programmzeilen nach dem Prompt (>>>) ein.

Merke: Der Index steht in eckigen Klammern. Der Index beginnt mit Null!

Notiere die print-Ausgaben in der Tabelle.

Eine Liste anlegen – mit eckigen Klammern!

Programmzeile	print-Ausgabe
<code>vornamen = ["Axel", "Elke", "Martin"]</code>	
<code>print(vornamen)</code>	
<code>print(vornamen[0])</code>	
<code>print(vornamen[0:2])</code>	
<code>print(vornamen[-1])</code>	
<code>vornamen[2] = "Fritz"</code>	
<code>print(vornamen)</code>	

Eine Liste erweitern

Programmzeile	print-Ausgabe
<code>vornamen = vornamen + ["Heike", "Sabine"]</code>	
<code>print(vornamen)</code>	
<code>vornamen += ["Markus"]</code>	
<code>print(vornamen)</code>	

Eine leere Liste anlegen und füllen

Programmzeile	print-Ausgabe
<code>buchstaben = []</code>	
<code>buchstaben.append("a")</code>	
<code>print(buchstaben)</code>	
<code>buchstaben.append("b")</code>	
<code>print(buchstaben)</code>	

Elemente einer Liste löschen

Programmzeile	print-Ausgabe
<code>print(vornamen)</code>	
<code>vornamen.remove("Heike")</code>	
<code>print(vornamen)</code>	
<code>del vornamen[0]</code>	
<code>print(vornamen)</code>	
<code>del vornamen</code>	
<code>print(vornamen)</code>	

Ein zufälliges Element aus einer Liste auswählen

Programmzeile	print-Ausgabe
<code>import random</code>	
<code>handzeichen = ["Schere", "Stein", "Papier"]</code>	
<code>print(random.choice(handzeichen))</code>	

Ein Tupel speichert viele nicht änderbare Elemente

Gib die Programmzeilen nach dem Prompt (>>>) ein.

Merke: Der Index steht in eckigen Klammern. Der Index beginnt mit Null!

Notiere die print-Ausgaben in der Tabelle.

Ein Tupel anlegen – mit runden Klammern!

Programmzeile	print-Ausgabe
<code>punkt = (-10, 5, 7)</code>	
<code>print(punkt)</code>	
<code>print(punkt[0])</code>	
<code>print(punkt[0:2])</code>	
<code>print(punkt[-1])</code>	
<code>punkt[2] = 15</code>	
<code>punkt = (-10, 5, 15)</code>	
<code>print(punkt)</code>	

Ein Element im Tupel suchen

Programmzeile	print-Ausgabe
<code>print(punkt)</code>	
<code>print(punkt.count(7))</code>	
<code>print(punkt.index(7))</code>	

Ein Dictionary speichert Paare von Schlüssel und Wert

Mit dem Menüpunkt File/ New File erzeugst du ein leeres Editor-Fenster mit dem Titel "untitled".

Gib die Programmzeilen im Editor-Fenster ein.

Speichere den Inhalt mit dem Menüpunkt File/ Save unter dem Dateinamen "woerterbuch" im Ordner Python/04_Listen_und_Woerterbuecher

Starte das Programm mit dem Menüpunkt Run/ Run Module.

Notiere die print-Ausgaben in der Tabelle.

Ein leeres dictionary anlegen – mit geschweiften Klammern – und füllen

Programmzeile	print-Ausgabe
# Wörterbuch Englisch - Deutsch	
# Leeres dictionary anlegen	
englisch_deutsch = {}	
# dictionary füllen	
englisch_deutsch["cat"] = "Katze"	
englisch_deutsch["dog"] = "Hund"	
englisch_deutsch["cow"] = "Kuh"	
print(englisch_deutsch)	
print(englisch_deutsch["dog"])	
englisch_deutsch["sheep"] = "Schaf"	
print(englisch_deutsch)	

Schlüssel und Werte des dictionary ausgeben

Programmzeile	print-Ausgabe
# Schlüssel ausgeben	
print(englisch_deutsch.keys())	
print(englisch_deutsch.values())	

Neues dictionary mit Vertauschung von Schlüssel und Wert erstellen

Wenn jeder Wert nur einmal im dictionary vorkommt, geht das mit folgender Programmzeile (Erläuterung später).

Programmzeile	print-Ausgabe
# neues dictionary erstellen	
deutsch_englisch = dict((v,k) for k,v in englisch_deutsch.items())	
print(deutsch_englisch)	
# Schlüssel ausgeben	
print(deutsch_englisch.keys())	
print(deutsch_englisch.values())	

Der Computer fragt ...

Mit dem Menüpunkt File/ New File erzeugst du ein leeres Editor-Fenster mit dem Titel "untitled".

Gib die Programmzeilen im Editor-Fenster ein.

Speichere den Inhalt mit dem Menüpunkt File/ Save unter dem Dateinamen "summe_ausgeben" im Ordner

Python/05_Benutzereingaben

Starte das Programm mit dem Menüpunkt Run/ Run Module.

Notiere die print-Ausgaben in der Tabelle.

Der Computer erwartet Zahlen

Programmzeile	print-Ausgabe
# Summe von zwei Zahlen ausgeben	
# Benutzereingaben anfordern	
zahl1 = input("Gib die erste Zahl ein ")	
zahl2 = input("Gib die zweite Zahl ein ")	
# Strings in Dezimalzahlen umwandeln	
zahl1 = float(zahl1)	
zahl2 = float(zahl2)	
# Summe ausgeben	
print("Die Summe der Zahlen ist", zahl1 + zahl2)	

Der Computer erwartet Strings

Programmzeile	print-Ausgabe
# Summe von zwei Strings ausgeben	
# Benutzereingaben anfordern	
str1 = input("Gib den ersten String ein ")	
str2 = input("Gib den zweiten String ein ")	
# Summe ausgeben	
print("Die Summe der Strings ist", str1 + str2)	

Aufgabe: Wörterbuch erweitern

Das Dictionary englisch_deutsch soll erweitert werden. Schreibe das Programm dazu.

- Lege das Dictionary englisch_deutsch an und fülle es mit 3 Paaren.
- Fordere ein neues englisches Wort an – den Schlüssel.
- Fordere das passende deutsche Wort an – den Wert.
- Erweitere das Dictionary mit Schlüssel und Wert.
- Drucke das erweiterte Dictionary

Was passiert, wenn der Schlüssel bereits vorhanden ist?

Der Computer unterscheidet zwischen "wahr" und "falsch"

Gib die Programmzeilen nach dem Prompt (>>>) ein.

Notiere die print-Ausgaben in der Tabelle.

Bedingung mit Zahlen

Programmzeile	print-Ausgabe
<code>print(1 == 2)</code>	
<code>print(1 != 2)</code>	
<code>print(1 < 2)</code>	
<code>print(1 > 2)</code>	
<code>print(3 == 3)</code>	
<code>print(3 != 3)</code>	
<code>print(3 <= 3)</code>	
<code>print(3 >= 3)</code>	

Bedingung mit Strings

Programmzeile	print-Ausgabe
<code>print("Hallo" == "Welt")</code>	
<code>print("Hallo" != "Welt")</code>	
<code>print("Montag" == "Montag")</code>	
<code>print("Montag" != "Montag")</code>	

Bedingung mit range(stop)

Programmzeile	print-Ausgabe
<code>bereich = range(10)</code>	
<code>print(list(bereich))</code>	
<code>print(1 in bereich)</code>	
<code>print(10 in bereich)</code>	

Bedingung mit range(start, stop)

Programmzeile	print-Ausgabe
<code>bereich = range(2, 10)</code>	
<code>print(list(bereich))</code>	
<code>print(1 in bereich)</code>	
<code>print(9 in bereich)</code>	

Bedingung mit range(start, stop, step)

Programmzeile	print-Ausgabe
<code>bereich = range(0, 10, 2)</code>	
<code>print(list(bereich))</code>	
<code>print(3 in bereich)</code>	
<code>print(8 in bereich)</code>	

Der Computer unterscheidet Fälle

Wenn die Bedingung "wahr" ist, werden die Programmzeilen darunter ausgeführt

Gib die Programmzeilen nach dem Prompt (>>>) ein.

Notiere die print-Ausgaben in der Tabelle.

Eine Bedingung – zwei Fälle

Programmzeile	print-Ausgabe
wert = 5	
if wert < 10:	
print("wert ist kleiner als 10")	
print("Ich gehöre auch zu der Bedingung")	

Eine Bedingung und die Alternative – zwei Fälle

Programmzeile	print-Ausgabe
if wert < 10:	
print("wert ist kleiner als 10")	
else:	
print("wert ist größer oder gleich 10")	

Mehrere Bedingungen und die Alternative – vier Fälle

Programmzeile	print-Ausgabe
if wert == 10:	
print("wert ist gleich 10")	
elif wert == 4:	
print("wert ist gleich 4")	
elif wert == 5:	
print("wert ist gleich 5")	
else:	
print("keine Bedingung ist erfüllt")	

Der Computer dreht Schleifen

Solange die Bedingung wahr ist, werden die Programmzeilen darunter ausgeführt

Mit dem Menüpunkt File/ New File erzeugst du ein leeres Editor-Fenster mit dem Titel "untitled".

Gib die Programmzeilen im Editor-Fenster ein.

Speichere den Inhalt mit dem Menüpunkt File/ Save unter dem Dateinamen "schleifen" im Ordner Python/07_Fallunterscheidungen_und_Schleifen

Starte das Programm mit dem Menüpunkt Run/ Run Module.

Notiere die print-Ausgaben in der Tabelle.

while Schleife

Programmzeile	print-Ausgabe
# Schleifen	
# while-Schleife	
# Variable initialisieren	
durchgang = 1	
while durchgang < 11:	
print(durchgang)	
durchgang = durchgang + 1	
print("nach der Schleife")	

Eine unendliche while Schleife abbrechen – break

Programmzeile	print-Ausgabe
# Variable initialisieren	
durchgang = 1	
# unendliche Schleife mit Abbruch	
while True:	
print("durchgang =", durchgang)	
durchgang = durchgang + 1	
if durchgang > 10:	
break	
print("Nach der Schleife")	

for Schleife – mit Liste

Programmzeile	print-Ausgabe
# Liste anlegen	
vornamen = ["Axel", "Elke", "Martin"]	
# Solange es ein Element in der Liste gibt	
for element in vornamen:	
print(element)	
print("nach der Schleife")	

for Schleife – mit range(stop)

Programmzeile	print-Ausgabe
# Solange es ein Element in der Liste gibt	
for element in range(10):	
print(element)	
print("nach der Schleife")	

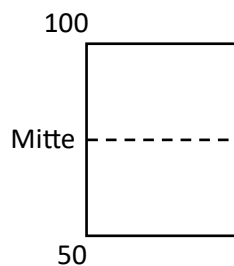
Aufgabe: Dialog mit dem Benutzer

Der Computer reagiert auf eine Benutzereingabe

Der Computer soll den Benutzer nach einem Buchstaben fragen und auf den Buchstaben reagieren. Schreibe das Programm dazu.

- Der Computer soll solange fragen, bis der Benutzer "e" eingibt.
- Jede Benutzereingabe soll gedruckt werden.

Die Mitte zwischen zwei Zahlen berechnen



Erweitere das Programm. Berechne die Mitte zwischen der unteren und der oberen Grenze.

- untere = 50 obere = 100
- Berechne und drucke die ganzzahlige Mitte zwischen "untere" und "obere"
- Teste deinen Ausdruck mit neuen Grenzen

Der Computer reagiert auf weitere Benutzereingaben

Erweitere das Programm.

- Der Computer soll solange fragen, bis der Benutzer "e" eingibt.
- Wenn der Benutzer "k" eingibt, soll "kleiner" gedruckt werden.
- Wenn der Benutzer "g" eingibt, soll "größer" gedruckt werden.

Funktionen haben Input und Output: Ganzzahlen

Funktionen mit Ganzzahlen (Integer) als Input und Output

Mit dem Menüpunkt File/ New File erzeugst du ein leeres Editor-Fenster mit dem Titel "untitled".

Gib die Programmzeilen im Editor-Fenster ein.

Speichere den Inhalt mit dem Menüpunkt File/ Save unter dem Dateinamen "funktionen_io" im Ordner Python/09_Funktionen_Input_und_Output

Starte das Programm mit dem Menüpunkt Run/ Run Module.

Notiere die print-Ausgaben in der Tabelle.

Funktion ohne Input

Programmzeile	print-Ausgabe
# Funktionen können Input und Output haben	
# Funktion ohne Input	
def ausgabe():	
print("hier bin ich")	
# Aufruf der Funktion	
ausgabe()	

Funktion mit 2 Inputs

Programmzeile	print-Ausgabe
# Funktion mit 2 Inputs	
def ausgabe2a(wert1: int, wert2: int):	
print("wert1 =", wert1, "wert2 =", wert2)	
# Aufruf der Funktion	
ausgabe2a(5, 6)	

Funktion mit 2 Inputs und Vorgabe

Die Werte mit Vorgabe stehen rechts von den Werten ohne Vorgabe.

Programmzeile	print-Ausgabe
# Funktion mit 2 Inputs und Vorgabe	
def ausgabe2b(wert1: int, wert2: int = 15):	
print("wert1 =", wert1, "wert2 =", wert2)	
# Aufruf der Funktion	
ausgabe2b(5)	

Funktion mit 1 Input und 1 Output

Programmzeile	print-Ausgabe
# Funktion mit 1 Input	
def verdoppeln(wert: int) -> int:	
return wert * 2	
# Aufruf der Funktion	
ergebnis = verdoppeln(5)	
print("ergebnis =", ergebnis)	

Funktionen haben Input und Output: Tupel, Liste

Funktion mit Tupel als Output

Programmzeile	print-Ausgabe
# Funktion mit Tupel als Output	
def wo_bin_ich() -> tuple[int, int]:	
x = 2	
y = 4	
return x, y	
# Aufruf der Funktion	
x, y = wo_bin_ich()	
print("x =", x, "y =", y)	

Funktion mit einer Liste als Input

Achtung: Eine Liste ist am Ort veränderbar (mutable object).

Input der Funktion ist eine Kopie der Liste, damit das Original unverändert bleibt.

Programmzeile	print-Ausgabe
# Funktion mit einer Liste als Input	
def verteuerung(liste: list[float], p:[float]):	
for i in range(len(liste)):	
liste[i] *= 1 + p	
# Liste anlegen	
original = [9, 12, 12.5, 24.5]	
# Liste kopieren	
kopie = original.copy()	
# Prozentsatz festlegen	
p = 0.05	
# Aufruf der Funktion	
verteuerung(kopie, p)	
# Original und Verteuerung	
print("original =", original)	
print("kopie =", kopie)	

Globale und lokale Variable in der Funktion

Ein Python Modul (eine Datei) hat einen eigenen privaten Namensraum. Alle Funktionen, die in diesem Modul definiert sind, nutzen ihn als globalen Namensraum.

Alle Variablen, die in einer Funktion nur gelesen werden, sind implizit global.

Beispiel: `bspfunktion_global_lesen()`

Wenn eine Variable in der Funktion einen Wert bekommt, ist sie lokal ...

Beispiel: `bspfunktion_lokal_schreiben()`

... wenn sie nicht explizit als global vereinbart wurde.

Beispiel: `bspfunktion_global_lesen_schreiben()`

```
1 # globale und lokale Variable in der Funktion
2
3 # Funktion liest eine globale Variable
4 def bspfunktion_global_lesen():
5     print("---- Funktion liest eine globale Variable ----")
6     # die gelesene Variable ist implizit global!
7     print("Variablenwert in Funktion:", variablenWert)
8
9 # Funktion gibt der Variable einen Wert
10 def bspfunktion_lokal_schreiben():
11     print("---- Funktion gibt der Variable einen Wert ----")
12     # die Variable bekommt einen Wert, damit ist sie lokal!
13     variablenWert = "IN der Funktion"
14     print("Variablenwert in Funktion:", variablenWert)
15
16 # Funktion liest eine globale Variable und gibt ihr einen Wert
17 def bspfunktion_global_lesen_schreiben():
18     # die Variable wird als global vereinbart
19     global variablenWert
20     print("---- Funktion liest eine globale Variable und gibt ihr einen Wert ----")
21     # die gelesene globale Variable
22     print("Variablenwert in Funktion 1:", variablenWert)
23     # die globale Variable bekommt einen Wert
24     variablenWert = "IN der Funktion"
25     print("Variablenwert in Funktion 2:", variablenWert)
26
27 # die Variable außerhalb der Funktion ist global
28 variablenWert = "außerhalb der Funktion"
29
30 print("\nVariablenwert vor dem Aufruf der Funktion:", variablenWert)
31 # Aufruf der Funktion
32 bspfunktion_global_lesen()
33 print("Variablenwert nach dem Aufruf der Funktion:", variablenWert)
34
35 print("\nVariablenwert vor dem Aufruf der Funktion:", variablenWert)
36 # Aufruf der Funktion
37 bspfunktion_lokal_schreiben()
38 print("Variablenwert nach dem Aufruf der Funktion:", variablenWert)
39
40 print("\nVariablenwert vor dem Aufruf der Funktion:", variablenWert)
41 # Aufruf der Funktion
42 bspfunktion_global_lesen_schreiben()
43 print("Variablenwert nach dem Aufruf der Funktion:", variablenWert)
```

Aufgaben mit Funktionen lösen: Konstruktionsanleitung

Die Konstruktionsanleitung hilft dabei:

1. Kurzbeschreibung
2. Datenanalyse
3. Funktion definieren: **Name** – **Input: Datentyp** – **Output: Datentyp**
4. Funktions-Rumpf
5. Ergebnisse prüfen
6. Unittest

Aufgabe: Sätze bauen

Gegeben sind drei Listen:

```
subjekt = ["Der Hund", "Die Journalistin", "Der Maler"]
```

```
prädikat = ["vergräbt", "interviewt", "malt"]
```

```
objekt = ["den Knochen", "den Bürgermeister", "ein Bild"]
```

Schreibe ein Programm, das ein zufälliges Subjekt und ein zufälliges Prädikat und ein zufälliges Objekt hintereinanderstellt und den zufälligen Satz ausgibt.

Starte das Programm und beurteile die print-Ausgabe.

Nr	Programmzeile
1	# Das Programm soll Subjekt, Prädikat, Objekt aus Listen
	# zufällig auswählen und einen Satz bauen
	# Bibliothek importieren
	import random
	# Beispielsätze
	subjekt = ["Der Hund", "Die Journalistin", "Der Maler"]
	prädikat = ["vergräbt", "interviewt", "malt"]
	objekt = ["den Knochen", "den Bürgermeister", "ein Bild"]
2	# Input der Funktion sind die Listen Subjekt, Prädikat und Objekt
	# Output der Funktion ist der Satz
	# Funktion mit Datentyp
3	def bau_den_satz(subjekt: list[str], prädikat: list[str], \
	objekt: list[str]) -> str:
4	mein_subjekt = random.choice(subjekt)
	mein_prädikat = random.choice(prädikat)
	mein_objekt = random.choice(objekt)
	mein_satz = mein_subjekt + " " + mein_prädikat + " " + mein_objekt
	return mein_satz
	# Funktion aufrufen
5	for i in range(3):
	mein_satz = bau_den_satz(subjekt, prädikat, objekt)
	# Ergebnis drucken
	print(mein_satz)

Aufgaben mit Funktionen lösen: Tabelle drucken

Aufgabe: Tabelle drucken

Wir wollen in Großbritannien einkaufen. Die Preise sind dort in britischen Pfund (GBP) angegeben. Wir müssen also umrechnen.

Schreibe ein Programm, das eine Umrechnungstabelle GBP in EUR druckt. Der Kurs ist: 1 GBP = 1,21 EUR

Die Tabelle soll von 0 GBP bis 10 GBP in Schritten von 0.50 GBP gehen.

Vervollständige das Programm.

Starte das Programm und beurteile die print-Ausgabe.

Nr	Programmzeile
1	# Das Programm soll GBP in EUR umrechnen und eine Tabelle ausgeben
2	# Input der Funktion ist eine Dezimalzahl in der Einheit GBP
	# Output der Funktion ist eine Dezimalzahl in der Einheit EUR
	# Umrechnungsfaktor 1 GBP = 1.21 EUR
	# Funktion mit Datentyp
3	Definiere die Funktion
4	Programmiere den Funktions-Rumpf
	# Input Liste anlegen
	gbp_liste = []
	for i in range(21):
	gbp_liste.append(i * 0.5)
	# Output Liste (leer) anlegen
	eur_liste = []
	# Funktion aufrufen
5	for x in gbp_liste:
	eur_liste.append(rufe die Funktion auf)
	# Ergebnisse drucken
	print("gbp eur")
	for x, y in zip(gbp_liste, eur_liste):
	print(x, y)

In der for-Schleife liefert zip(gbp_liste, eur_liste) ein Tupel mit einem Element aus jeder Liste.

Aufgaben mit Funktionen lösen: Diagramm plotten

Aufgabe: Tabelle drucken und Diagramm plotten

Unsere Stromkosten sind hoch. Deshalb denken wir über einen Wechsel des Stromanbieters nach.

Angebot	Grundgebühr pro Monat	Verbrauchspreis pro kWh
Stromtarif "Watt für wenig"	15,60 €	0,32 €
Stromtarif "Billig Strom"	12,80 €	0,36 €

Welches Angebot ist günstiger? Das hängt von unserem monatlichen Stromverbrauch ab.

Schreibe ein Programm, das eine Vergleichstabelle druckt. Die Überschrift ist:

Verbrauch Watt für wenig Billig Strom

Darunter stehen der monatliche Verbrauch und die berechneten monatlichen Kosten der beiden Angebote.

Der Verbrauch geht von 0 kWh bis 150 kWh in Schritten von 10 kWh. Zeige den Vergleich auch in einem Diagramm.

Vervollständige das Programm. Starte das Programm und beurteile die print-Ausgabe.

Nr	Programmzeile
1	# Programm vergleicht die Kosten von zwei Stromtarifen
2	# 1. Angebot: Input der Funktion ist eine Dezimalzahl in der Einheit kWh # Output der Funktion ist eine Dezimalzahl in der Einheit EUR # Funktion mit Datentyp
3	Definiere die Funktion
4	Programmiere den Funktions-Rumpf
2	# 2. Angebot: Input der Funktion ist eine Dezimalzahl in der Einheit kWh # Output der Funktion ist eine Dezimalzahl in der Einheit EUR # Funktion mit Datentyp
3	Definiere die Funktion
4	Programmiere den Funktions-Rumpf
	# Input Liste anlegen
	Lege die Liste an
	# 1. Angebot: Output Liste (leer) anlegen
	Lege die Liste an
	# 2. Angebot: Output Liste (leer) anlegen
	Lege die Liste an
	# Funktionen aufrufen
5	Programmiere eine Schleife
	Fülle die Liste
	Fülle die Liste
	# Ergebnisse drucken
	Drucke die Überschrift
	Programmiere eine Schleife
	Drucke das Ergebnis
	# Modul für das Plotten von Graphen importieren
	import matplotlib.pyplot as plt
	# Ergebnisse plotten
5	plt.plot(verbrauch, kosten1)
	plt.plot(verbrauch, kosten2)
	plt.xlabel("Verbrauch")
	plt.ylabel("monatliche Kosten")
	plt.show()

Klassen haben Eigenschaften und Methoden

Eine Instanz der Klasse beschreibt ein konkretes Objekt mit Eigenschaften und Methoden

Aufgabe: Eine Klasse und eine Instanz programmieren

Programmiere die Klasse "Fahrrad" mit den Eigenschaften:

Besitzer, Farbe, Typ (Touring, Renn, Mountain), Anzahl Gänge

Programmiere die Methoden:

klingeln, fahren

Erstelle zwei Instanzen der Klasse "Fahrrad" und gib die Eigenschaften aus.

Rufe dann die Methoden der Instanzen auf.

Das Programm Katzen_Klasse.py löst eine ganz ähnliche Aufgabe.

Schreibe das Programm Fahrrad_Klasse.py nach dem Vorbild Katzen_Klasse.py.

Die Vergleichstabelle hilft dabei.

Starte das Programm und beurteile die print-Ausgabe.

Katzen_Klasse.py	Fahrrad_Klasse.py
<code>class BauplanKatzenKlasse</code>	<code>class BauplanFahrradKlasse</code>
<code>__init__(self, rufname, farbe, alter)</code>	<code>__init__(self, besitzer, farbe, typ, gaenge)</code>
<code>rufname</code>	<code>besitzer</code>
<code>farbe</code>	<code>farbe</code>
	<code>typ</code>
<code>alter</code>	<code>gaenge</code>
<code>schlafdauer</code>	<code>kmstand</code>
<code>tut_miauen</code>	<code>klingeln</code>
<code>tut_schlafen</code>	<code>fahren</code>
<code>katze_sammy</code>	<code>mein_fahrrad</code>
<code>katze_sammy.rufname</code>	<code>mein_fahrrad.besitzer</code>
<code>katze_sammy.farbe</code>	<code>mein_fahrrad.farbe</code>
	<code>mein_fahrrad.typ</code>
<code>katze_sammy.alter</code>	<code>mein_fahrrad.gaenge</code>
<code>katze_soni</code>	<code>leih_fahrrad</code>
<code>katze_sammy.tut_miauen</code>	<code>mein_fahrrad.klingeln</code>
<code>katze_sammy.tut_schlafen</code>	<code>mein_fahrrad.fahren</code>
<code>katze_soni.tut_schlafen</code>	<code>leih_fahrrad.fahren</code>

Die Eltern-Klasse vererbt – die Kind-Klasse erbt

Die Kind-Klasse erbt alle Eigenschaften und Methoden der Eltern-Klasse.

Aufgabe: Eine Eltern-Klasse und zwei Kind-Klassen programmieren

Erstelle die Eltern-Klasse "Zweirad" mit den Eigenschaften: Besitzer, Farbe, Typ, Anzahl Gänge und den Methoden: fahren, klingeln.

Erstelle die Kind-Klassen "Fahrrad" und "Pedelec", die alle Eigenschaften und Methoden der Eltern-Klasse "Zweirad" erben.

Die Klasse "Pedelec" hat zusätzlich die Eigenschaft "Kapazität" (Wattstunden) und die Methode "aufladen".

Erstelle eine Instanz der Klasse "Fahrrad" und eine Instanz der Klasse "Pedelec" und rufe alle Methoden auf. Gib die Eigenschaft "Kapazität" der Instanz des "Pedelec" aus.

Das Programm Tier_Klasse.py löst eine ganz ähnliche Aufgabe.

Schreibe das Programm Zweirad_Klasse.py nach dem Vorbild Tier_Klasse.py.

Die Vergleichstabelle hilft dabei.

Starte das Programm und beurteile die print-Ausgabe.

Tier_Klasse.py	Zweirad_Klasse.py
<code>class Tier()</code>	<code>class Zweirad()</code>
<code>__init__(self, rufname, farbe, alter)</code>	<code>__init__(self, besitzer, farbe, typ, gaenge)</code>
<code>rufname</code>	<code>besitzer</code>
<code>farbe</code>	<code>farbe</code>
	<code>typ</code>
<code>alter</code>	<code>gaenge</code>
<code>schlafdauer</code>	<code>kmstand</code>
<code>tut_reden</code>	<code>klingeln</code>
<code>tut_schlafen</code>	<code>fahren</code>
<code>class BauplanKatzenKlasse(Tier)</code>	<code>class BauplanFahrradKlasse(Zweirad)</code>
<code>__init__(self, rufname, farbe, alter)</code>	<code>__init__(self, besitzer, farbe, typ, gaenge)</code>
<code>super.__init__(self, rufname, farbe, alter)</code>	<code>super.__init__(self, besitzer, farbe, typ, gaenge)</code>
<code>class Hund(Tier)</code>	<code>class Pedelec(Zweirad)</code>
<code>__init__(self, rufname, farbe, alter)</code>	<code>__init__(self, besitzer, farbe, typ, gaenge, kapazitaet)</code>
<code>super.__init__(self, rufname, farbe, alter)</code>	<code>super.__init__(self, besitzer, farbe, typ, gaenge, kapazitaet)</code>
	<code>kapazitaet</code>
<code>tut_reden</code>	<code>aufladen</code>
<code>katze sammy</code>	<code>mein_fahrrad</code>
<code>katze sammy.farbe</code>	<code>mein_fahrrad.besitzer</code>
<code>hund bello</code>	<code>dein_pedelec</code>
<code>hund bello.farbe</code>	<code>dein_pedelec.besitzer</code>
	<code>dein_pedelec.kapazitaet</code>
<code>hund bello.tut_schlafen</code>	<code>dein_pedelec.fahren</code>
<code>katze sammy.tut_schlafen</code>	<code>mein_fahrrad.fahren</code>
<code>katze sammy.tut_reden</code>	<code>mein_fahrrad.klingeln</code>
<code>hund bello.tut_reden</code>	<code>dein_pedelec.klingeln</code>
	<code>dein_pedelec.aufladen</code>

Roboter im Irrgarten: Wände bauen

Wir arbeiten mit der grafischen Benutzeroberfläche "Turtle"

Aufgabe: Einen Irrgarten bauen – Irrgarten_Klasse.py

Das Programm bringt ein Fenster mit 3 Blöcken unterschiedlicher Farbe auf den Bildschirm. Starte das Programm. Ändere das Programm und starte es erneut.

Aufgabe

Setze Blöcke nebeneinander und untereinander, um Wände zu bauen. Experimentiere mit den Farben.

```

1 # Programm erzeugt ein Fenster und setzt die Wände eines Irrgartens hinein
2 # - Eine Funktion setzt die Wände in das Fenster
3 # Modul für die Turtle-Grafik importieren
4 import turtle
5 # Irrgarten-Klasse
6 class Maze:
7     """Klasse für den Bau eines Irrgartens"""
8     # Methoden der Klasse
9     def __init__(self):
10         self.rows_in_maze = 11
11         self.columns_in_maze = 22
12         # turtle Objekt erzeugen und Aussehen festlegen
13         self.t = turtle.Turtle()
14         self.t.shape("turtle")
15         # Breite, Höhe und Koordinaten des Fensters festlegen
16         self.wn = turtle.Screen()
17         self.wn.setup(800, 400)
18         self.wn.setworldcoordinates(0, 0, self.columns_in_maze, self.rows_in_maze)
19     # Zeichne ein ausgefülltes Rechteck
20     def draw_box(self, x, y, color):
21         self.t.up() # Stift hoch
22         self.t.goto(x, y)
23         self.t.color(color)
24         self.t.fillcolor(color)
25         self.t.setheading(90)
26         self.t.down() # Stift runter
27         self.t.begin_fill()
28         # Rechteck zeichnen und füllen
29         for i in range(4):
30             self.t.forward(1)
31             self.t.right(90)
32         self.t.end_fill()
33     # Setze Wände in das Fenster
34     def draw_maze(self):
35         # Farben: 'white', 'black', 'red', 'green', 'blue', 'cyan', 'yellow', 'magenta'
36         self.draw_box(0, 0, "orange")
37         self.draw_box(self.columns_in_maze - 1, self.rows_in_maze - 1, "magenta")
38         self.draw_box(self.columns_in_maze//2, self.rows_in_maze//2, "cyan")
39         # Farbe der Schildkröte
40         self.t.color("black")
41         self.t.fillcolor("blue")
42         self.wn.update()
43     # Instanz erzeugen
44     my_maze = Maze()
45     # Irrgarten zeichnen
46     my_maze.draw_maze()
47     my_maze.t.up() # Stift hoch
48     # Schildkröte in Home-Position
49     my_maze.t.home()
50     # Turtle event loop
51     my_maze.wn.mainloop()

```


Roboter im Irrgarten: Wände ablegen

Aufgabe: Wände des Irrgartens in einer Liste ablegen - Irrgarten_Klasse_maze_list.py

Überall wo ein '+' in der Liste steht, zeichnet das Programm einen Block in das Fenster. In der Mitte des Fensters formen die Wände ein Wort. Starte das Programm. Ändere das Programm und starte es erneut.

Aufgabe	Ergebnis
Ersetze in Zeile 62 <code>self.from_bottom(row)</code> durch <code>row</code>	
Kommentiere die Zeile 58 aus	

[illegible]

Roboter im Irrgarten: Wände aus Datei lesen

Aufgabe: Wände des Irrgartens aus einer Datei lesen - Irrgarten_Klasse_maze_filename.py

Das Programm öffnet die Datei `maze3.txt` und kopiert alle Zeilen in der Datei in die Variable `lines`.

Dann geht das Programm durch jede Zeile von `lines` und erzeugt eine Liste mit allen Zeichen der Zeile.

Diese Liste wird an `maze_list` angehängt. Starte das Programm.

Ändere die Datei `maze3.txt` und starte das Programm erneut.

Aufgabe	Ergebnis
Vergleiche die '+' in <code>maze3.txt</code> mit den Blöcken im Fenster	
Ergänze und entferne '+' in <code>maze3.txt</code> und vergleiche erneut	

Inhalt der Datei `maze3.txt`

```

+++++
+      ++ ++  +  +
+      +++++ ++ +  +++  +
+      +      +  +      +
+ +++++      +  +  +++  +
+      +      +  +++  +
+ +++++++      +      +
+      +      +++++  +
+ ++++++++      +  +  +
+      S  +  +
+++++

```

```

10 # Irrgarten-Klasse
11 class Maze:
12     """Klasse für den Bau eines Irrgartens"""
13     # Methoden der Klasse
14     def __init__(self, maze_filename):
15         # maze_list aus Datei lesen
16         self.maze_list = []
17         with open(maze_filename, "r") as maze_file:
18             self.lines = maze_file.readlines()
19             for line in self.lines:
20                 self.maze_list.append([ch for ch in line.rstrip("\n")])
21             self.rows_in_maze = len(self.maze_list)
22             self.columns_in_maze = len(self.maze_list[0])
23             print("rows in maze =", self.rows_in_maze, "columns in maze =", self.columns_in_maze)
24 # Instanz erzeugen
25 my_maze = Maze("maze3.txt")
26 # Irrgarten zeichnen
27 my_maze.draw_maze()
28 my_maze.t.up() # Stift hoch
29 # Schildkröte in Home-Position
30 my_maze.t.home()
31 # Turtle event loop
32 my_maze.wn.mainloop()

```

Roboter im Irrgarten: Schildkröte bewegen

Aufgabe: Schildkröte durch den Irrgarten bewegen - Irrgarten_Klasse_move_turtle.py

Nachdem das Programm die Liste `maze_list` erstellt hat, sucht es in `maze_list` nach dem Startpunkt "S".

Die Funktion `search_from()` setzt die Schildkröte auf den Startpunkt. Starte das Programm. Ändere das Programm und starte es erneut.

Aufgabe	Ergebnis
Vergleiche das 'S' in <code>maze3.txt</code> mit der Position der Schildkröte	
Schreibe hinter die Zeile 89: <code>maze.update_position(start_row - 2, start_col)</code>	

```

12 # Irrgarten-Klasse
13 class Maze:
14     """Klasse für den Bau eines Irrgartens"""
15     # Methoden der Klasse
16     def __init__(self, maze_filename):
17         # maze_list aus Datei lesen
18         self.maze_list = []
19         with open(maze_filename, "r") as maze_file:
20             self.lines = maze_file.readlines()
21             for line in self.lines:
22                 self.maze_list.append([ch for ch in line.rstrip("\n")])
23         self.rows_in_maze = len(self.maze_list)
24         self.columns_in_maze = len(self.maze_list[0])
25         print("rows_in_maze =", self.rows_in_maze, "columns_in_maze =", self.columns_in_maze)
26         # Start in maze_list suchen
27         for row in range(self.rows_in_maze):
28             for col in range(self.columns_in_maze):
29                 if self.maze_list[row][col] == START:
30                     self.start_row = row
31                     self.start_col = col
32                     break
33         print("start_row =", self.start_row, "start_col =", self.start_col)
34
35     # Bewege die Schildkröte
36     def move_turtle(self, x, y):
37         x += 0.5
38         y += 0.5
39         self.t.setheading(self.t.towards(x, y))
40         self.t.goto(x, y)
41
42     # Aktualisiere die Position der Schildkröte
43     def update_position(self, row, col, val=None):
44         # Wenn val angegeben: Element der Liste überschreiben
45         if val:
46             self.maze_list[row][col] = val
47             self.move_turtle(col, self.from_bottom(row))
48
49 # Roboter sucht den Weg, Schildkröte läuft mit
50 def search_from(maze, start_row, start_col):
51     # Schildkröte auf die Startposition setzen
52     maze.update_position(start_row, start_col, BLANK)
53     # Der Roboter sucht den Weg ... folgt
54
55 # Instanz erzeugen
56 my_maze = Maze("maze3.txt")
57 # Irrgarten zeichnen
58 my_maze.draw_maze()
59 my_maze.t.up() # Stift hoch
60 # Schildkröte in Home-Position
61 my_maze.t.home()
62 # Suche den Weg vom Start zum Ausgang
63 search_from(my_maze, my_maze.start_row, my_maze.start_col)
64
65 # Turtle event loop
66 my_maze.wn.mainloop()

```

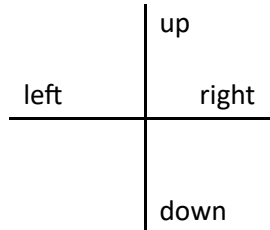
Roboter im Irrgarten: Schauen, gehen, drehen

Aufgabe: Roboter schaut, geht und dreht - Irrgarten_Klasse_robot_methods.py

Das Programm kann die Schildkröte bewegen. Die Schildkröte erkennt aber keine Wände. Wir benötigen also einen Roboter, der schaut, geht und dreht. Die Position des Roboters wird durch seine `row` und `col` dargestellt.

Der Roboter soll Folgendes können:

- in alle 4 Richtungen schauen
- in alle 4 Richtungen gehen
- nach rechts drehen
- nach links drehen



Rechts und links sind abhängig von der aktuellen Richtung. Ein Dictionary gibt uns die neue Richtung für rechts und für links. Beispiel: Die aktuelle Richtung ist "down". Rechts ist "left".

In eine Richtung schauen oder gehen bedeutet:

- `row` unverändert oder `row + 1` oder `row - 1` *und*
- `col` unverändert oder `col + 1` oder `col - 1`

Ein Dictionary gibt uns das für `row` und `col` vor. Beispiel: "left" bedeutet `row` unverändert *und* `col - 1`.

Die Klasse `Maze` bekommt zusätzlich 6 Roboter-Methoden. Wir testen die Methoden mit Hilfe der Funktion `search_from()`. Die Funktion enthält eine Benutzerabfrage. Dort gebt ihr ein Kommando ein, und der Roboter schaut, geht und dreht. Starte das Programm. Gib Kommandos ein und beobachte die Reaktion von Roboter und Schildkröte. Tipp: Das Kommando `h[and]` steht für: Schaue zur rechten Hand.

Aufgabe	Ergebnis
Gib die Kommandos <code>v[orne]</code> , <code>h[and]</code> , <code>l[inks]</code> , <code>r[echts]</code> , <code>s[chritt]</code> , <code>e[nde]</code> ein und bewege den Roboter eine kurze Strecke durch den Irrgarten. Kann der Roboter durch Wände gehen?	

```

14 # Richtungs-Eigenschaften
15 right_of = {"up": "right", "down": "left", "left": "up", "right": "down"}
16 left_of  = {"up": "left", "down": "right", "left": "down", "right": "up"}
17 angle_of = {"up": 90, "down": 270, "left": 180, "right": 0}
18 delta_row = {"up": -1, "down": 1, "left": 0, "right": 0}
19 delta_col = {"up": 0, "down": 0, "left": -1, "right": 1}

94 # Exit erreicht?
95 def is_exit(self, row, col):
96     return (
97         row == 0
98         or row == self.rows_in_maze - 1
99         or col == 0
100        or col == self.columns_in_maze - 1
101    )
102 # Geschwindigkeit und Richtung der Schildkröte einstellen
103 def init_search(self, heading):
104     self.t.speed(3)
105     self.t.pendown() # Stift runter
106     self.t.setheading(angle_of[heading])
107 # schau nach vorne
108 def look_forward(self, start_row, start_col, heading):
109     # im Exit nicht nach vorne schauen!
110     if self.is_exit(start_row, start_col) == True:
111         return EXIT
112     else:
113         return self.maze_list[start_row + delta_row[heading]]\
114             [start_col + delta_col[heading]]
115 # schau nach rechts
116 def look_right(self, start_row, start_col, heading):
117     return self.maze_list[start_row + delta_row[right_of[heading]]]\
118         [start_col + delta_col[right_of[heading]]]

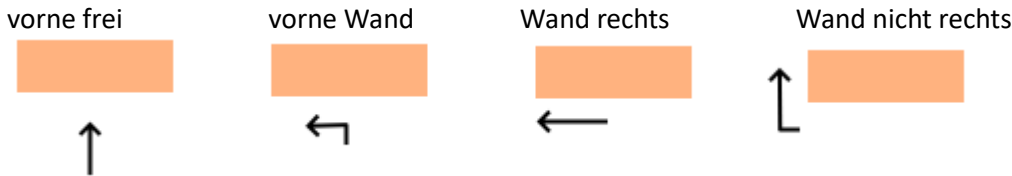
```

```
119     # mache einen Schritt
120     def one_step(self, start_row, start_col, heading):
121         start_row += delta_row[heading]
122         start_col += delta_col[heading]
123         return start_row, start_col
124     # drehe dich nach rechts
125     def turn_right(self, heading):
126         self.t.right(90)
127         return right_of[heading]
128     # drehe dich nach links
129     def turn_left(self, heading):
130         self.t.left(90)
131         return left_of[heading]
132 # Roboter sucht den Weg, Schildkröte läuft mit
133 def search_from(maze, start_row, start_col):
134     # Schildkröte auf die Startposition setzen
135     maze.update_position(start_row, start_col, BLANK)
136     # Richtung festlegen
137     heading = "up"
138     print(heading)
139     # Schildkröte einstellen
140     maze.init_search(heading)
141     # cmd initialisieren
142     cmd = ""
143     # Solange wiederholen, bis Benutzer "e" eingibt
144     while cmd != "e":
145         # cmd leeren
146         cmd = ""
147         # Solange wiederholen, bis Benutzer etwas eingibt
148         while cmd == "":
149             cmd = input("Kommando v[orne], h[and], l[inks], r[echts], s[chritt], e[nde] ")
150             print("Kommando =", cmd)
151             if maze.is_exit(start_row, start_col) == True:
152                 print("Turtle im Exit!")
153                 print("Ende - du kannst das Turtle-Fenster jetzt schließen")
154                 break
155             elif cmd == "v":
156                 ch = maze.look_forward(start_row, start_col, heading)
157                 print(ch)
158             elif cmd == "h":
159                 ch = maze.look_right(start_row, start_col, heading)
160                 print(ch)
161             elif cmd == "l":
162                 heading = maze.turn_left(heading)
163                 print(heading)
164             elif cmd == "r":
165                 heading = maze.turn_right(heading)
166                 print(heading)
167             elif cmd == "s":
168                 start_row, start_col = maze.one_step(start_row, start_col, heading)
169                 maze.update_position(start_row, start_col)
170                 print("start_row =", start_row, "start_col =", start_col)
171             elif cmd == "e":
172                 print("Ende - du kannst das Turtle-Fenster jetzt schließen")
173             else:
174                 print("Unbekanntes Kommando")
175     # Der Roboter sucht den Weg ... folgt
```

Roboter im Irrgarten: Rechte-Hand-Methode

Aufgabe: Mit der Rechte-Hand-Methode findet der Roboter aus dem Irrgarten -
 turtle_in_maze_right_hand_rule_2x1.py

Die Methoden `look_forward()` und `look_right()` melden eine der folgenden Situationen.



Der Pfeil zeigt, wie der Roboter reagieren soll, damit seine rechte Hand an der Wand bleibt.

Der Roboter hat den Ausgang erreicht, wenn die Methode `is_exit()` `True` meldet. Starte das Programm.

Aufgabe	Ergebnis
Ersetze nun in <code>maze3.txt</code> Zeile 8 Spalte 17 das <code>+</code> durch ein Leerzeichen. Findet der Roboter den Weg aus dem Irrgarten? Wie muss der Irrgarten beschaffen sein, damit der Roboter hinausfindet?	

```

131 def search_from(maze, start_row, start_col):
132     # Schildkröte auf die Startposition setzen
133     maze.update_position(start_row, start_col, BLANK)
134     # Richtung festlegen
135     heading = "up"
136     print(heading)
137     # Schildkröte einstellen
138     maze.init_search(heading)
139
140     # solange vorne frei ist
141     while maze.look_forward(start_row, start_col, heading) == BLANK:
142         start_row, start_col = maze.one_step(start_row, start_col, heading)
143         maze.update_position(start_row, start_col)
144     # drehen, damit rechte Hand an der Wand ist
145     heading = maze.turn_left(heading)
146
147     # Drehungen zählen
148     turn_count = 1
149
150     # Folge der Wand bis Exit
151     while maze.is_exit(start_row, start_col) == False:
152
153         # solange vorne frei und die Wand rechts ist
154         while maze.look_forward(start_row, start_col, heading) == BLANK\
155             and maze.look_right(start_row, start_col, heading) == OBSTACLE:
156             start_row, start_col = maze.one_step(start_row, start_col, heading)
157             maze.update_position(start_row, start_col)
158
159         # wenn Exit, dann Schleife abbrechen
160         if maze.is_exit(start_row, start_col) == True:
161             break
162
163         # wenn die Wand nicht mehr rechts ist
164         elif maze.look_right(start_row, start_col, heading) == BLANK:
165             # drehen und 1 Schritt vorwärts, damit rechte Hand an der Wand ist
166             heading = maze.turn_right(heading)
167             turn_count += 1
168             start_row, start_col = maze.one_step(start_row, start_col, heading)
169             maze.update_position(start_row, start_col)
170
171         # wenn vorne eine Wand ist
172         elif maze.look_forward(start_row, start_col, heading) == OBSTACLE:
173             # drehen, damit rechte Hand an der Wand ist
174             heading = maze.turn_left(heading)
175             turn_count += 1
176
177     # Ende der while-Schleife
178     print("Exit found!")
179     print(turn_count, "Drehungen")
  
```

Roboter im Irrgarten: Pledge-Algorithmus

Aufgabe: Mit dem Pledge-Algorithmus findet der Roboter aus jedem Irrgarten - `turtle_in_maze_pledge_algorithm_2x1.py`

Der Pledge-Algorithmus arbeitet mit dem Drehungs-Level `turn_level`. Zu Beginn ist der Drehungs-Level Null. Bei einer Linksdrehung wird er um 1 erhöht, bei einer Rechtsdrehung um 1 erniedrigt. Der Roboter läuft geradeaus bis zur Wand und macht eine Linksdrehung, damit die rechte Hand an die Wand ist. Danach folgt er der Wand bis zum Ausgang oder bis der Drehungs-Level Null ist. Wenn der Drehungs-Level Null ist, läuft der Roboter wieder geradeaus bis zur Wand, ohne auf die rechte Hand zu achten. Starte das Programm.

Aufgabe	Ergebnis
Ersetze nun in <code>maze3.txt</code> Zeile 8 Spalte 17 das + durch ein Leerzeichen. Findet der Roboter den Weg aus dem geänderten Irrgarten <code>maze3.txt</code> ?	
Teste das Programm mit dem Irrgarten <code>my_maze.txt</code>	

```

131 def search_from(maze, start_row, start_col):
132     # Schildkröte auf die Startposition setzen
133     maze.update_position(start_row, start_col, BLANK)
134     # Richtung festlegen
135     heading = "up"
136     print(heading)
137     # Schildkröte einstellen
138     maze.init_search(heading)
139
140     # Drehungen zählen
141     turn_count = 0
142     # Drehungs-Level berechnen: Linksdrehung +1, Rechtsdrehung -1
143     turn_level = 0
144

```

```
145 # Wiederhole bis Exit
146 while maze.is_exit(start_row, start_col) == False:
147
148     # solange vorne frei ist
149     print("turn_level =", turn_level)
150     while maze.look_forward(start_row, start_col, heading) == BLANK:
151         start_row, start_col = maze.one_step(start_row, start_col, heading)
152         maze.update_position(start_row, start_col)
153     # drehen, damit rechte Hand an der Wand ist
154     heading = maze.turn_left(heading)
155     turn_count += 1
156     turn_level += 1
157
158     # wenn Exit, dann Schleife abbrechen
159     if maze.is_exit(start_row, start_col) == True:
160         break
161
162     # solange der Drehungs-Level ungleich Null ist
163     while turn_level != 0:
164
165         # solange vorne frei und die Wand rechts ist
166         print("turn_level =", turn_level)
167         while maze.look_forward(start_row, start_col, heading) == BLANK\
168             and maze.look_right(start_row, start_col, heading) == OBSTACLE:
169             start_row, start_col = maze.one_step(start_row, start_col, heading)
170             maze.update_position(start_row, start_col)
171
172         # wenn Exit, dann Schleife abbrechen
173         if maze.is_exit(start_row, start_col) == True:
174             break
175
176         # wenn die Wand nicht mehr rechts ist
177         elif maze.look_right(start_row, start_col, heading) == BLANK:
178             # drehen und 1 Schritt vorwärts, damit rechte Hand an der Wand ist
179             heading = maze.turn_right(heading)
180             turn_count += 1
181             turn_level -= 1
182             start_row, start_col = maze.one_step(start_row, start_col, heading)
183             maze.update_position(start_row, start_col)
184
185         # wenn vorne eine Wand ist
186         elif maze.look_forward(start_row, start_col, heading) == OBSTACLE:
187             # drehen, damit rechte Hand an der Wand ist
188             heading = maze.turn_left(heading)
189             turn_count += 1
190             turn_level += 1
191
192     # Ende äußeren while-Schleife
193     print("Exit found!")
194     print(turn_count, "Drehungen")
```


Auto im Gegenverkehr: Auto steuern

Wir arbeiten mit der Bibliothek "Pygame" für Computerspiele

Aufgabe: Wir steuern das Auto nach links – nach rechts – vorwärts – rückwärts –
Spieler_Klasse_Auto_steuern.py

Das Programm bringt ein Fenster mit einem blauen Auto auf den Bildschirm. Starte das Programm. Ändere das Programm und starte es erneut.

Aufgabe	Ergebnis
Steuere das Auto mit den Pfeiltasten der Tastatur. Warum verlässt das Auto das Fenster nicht?	
Ersetze das blaue Auto durch das größere gelbe Auto. Verlässt das gelbe Auto das Fenster?	

```

1| # Programm erzeugt ein fenster mit einem Auto
2| # - Das Auto wird mit den Pfeil-Tasten gesteuert
3| # Bibliotheken importieren
4| import pygame, sys
5| # alle Module initialisieren
6| pygame.init()
7| # Frames per second festlegen
8| FPS = 60
9| # Clock objekt erzeugen
10| FramePerSec = pygame.time.Clock()
11| # Breite und Höhe des Fensters festlegen
12| SCREEN_WIDTH = 400
13| SCREEN_HEIGHT = 600
14| # Farbe definieren
15| WHITE = (255, 255, 255)
16| # Grafik-Fenster erzeugen
17| screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
18| screen.fill(WHITE)
19| # Spieler-Klasse
20| class Player(pygame.sprite.Sprite):
21|     """Klasse für den Spieler"""
22|     def __init__(self):
23|         super().__init__()
24|         # Bild laden
25|         self.image = pygame.image.load("blue_car.png")
26|         # Rechteck in der Größe des Bildes erzeugen
27|         self.rect = self.image.get_rect()
28|         # Anfangsposition definieren
29|         self.rect.center = (160, 520)
30| # Player bewegen
31| def move(self):
32|     # Zustand aller Tasten abfragen
33|     pressed_keys = pygame.key.get_pressed()
34|     # Wenn noch im Fenster: Player mit Taste steuern
35|     if self.rect.left > 0:
36|         if pressed_keys[pygame.K_LEFT]:
37|             self.rect.move_ip(-5, 0)
38|     if self.rect.right < SCREEN_WIDTH:
39|         if pressed_keys[pygame.K_RIGHT]:
40|             self.rect.move_ip(5, 0)
41|     if self.rect.top > 0:
42|         if pressed_keys[pygame.K_UP]:
43|             self.rect.move_ip(0, -5)
44|     if self.rect.bottom < SCREEN_HEIGHT:
45|         if pressed_keys[pygame.K_DOWN]:
46|             self.rect.move_ip(0, 5)

```



```
47 # Instanz erzeugen
48 P1 = Player()
49 #Game Loop
50 while True:
51     # Ereignisse prüfen
52     for event in pygame.event.get():
53         if event.type == pygame.QUIT:
54             # pygame Fenster schließen
55             pygame.quit()
56             # Python Skript beenden
57             sys.exit()
58     # Player bewegen
59     P1.move()
60     # Grafik-Fenster auswischen
61     screen.fill(WHITE)
62     # Player zeichnen
63     # Blocktransfer: source = P1.image, destination = P1.rect
64     screen.blit(P1.image, P1.rect)
65     # Grafik-Fenster aktualisieren
66     pygame.display.update()
67     # Frames per second begrenzen
68     FramePerSec.tick(FPS)
```

Auto im Gegenverkehr: Gegenverkehr kommt

Aufgabe: Ein Auto kommt entgegen – Spieler_Klasse_Gegner_Klasse_Gegenverkehr.py

Das Programm bringt ein Fenster mit einem blauen Auto *und einem entgegenkommenden roten Auto* auf den Bildschirm. Starte das Programm. Ändere das Programm und starte es erneut.

Aufgabe	Ergebnis
Steuere das Auto mit den Links- und Rechts-Pfeiltasten der Tastatur. Was passiert bei einem Zusammenstoß der beiden Autos?	
Verdopple die Geschwindigkeit des roten Autos. Die Geschwindigkeit des blauen Autos soll dabei unverändert bleiben.	

```

43 # Gegner-Klasse
44 class Enemy(pygame.sprite.Sprite):
45     def __init__(self):
46         super().__init__()
47         # Bild laden
48         self.image = pygame.image.load("red_car.png")
49         # Rechteck in der Größe des Bildes erzeugen
50         self.rect = self.image.get_rect()
51         # zufällige Anfangsposition
52         self.rect.center = (random.randint(40, SCREEN_WIDTH-40), 0)
53     # Enemy bewegen
54     def move(self):
55         self.rect.move_ip(0,10)
56         # Wenn im Fenster unten angekommen
57         if (self.rect.bottom > SCREEN_HEIGHT):
58             # oben im Fenster beginnen
59             self.rect.top = 0
60             # zufällige Anfangsposition
61             self.rect.center = (random.randint(40, SCREEN_WIDTH - 40), 0)
62 # Instanzen erzeugen
63 P1 = Player()
64 E1 = Enemy()
65 #Game Loop
66 while True:
67     # Ereignisse prüfen
68     for event in pygame.event.get():
69         if event.type == pygame.QUIT:
70             # pygame Fenster schließen
71             pygame.quit()
72             # Python Skript beenden
73             sys.exit()
74     # Player bewegen
75     P1.move()
76     # Enemy bewegen
77     E1.move()
78     # Grafik-Fenster auswischen
79     screen.fill(WHITE)
80     # Player zeichnen
81     # Blocktransfer: source = P1.image, destination = P1.rect
82     screen.blit(P1.image, P1.rect)
83     # Enemy zeichnen
84     # Blocktransfer: source = E1.image, destination = E1.rect
85     screen.blit(E1.image, E1.rect)
86     # Grafik-Fenster aktualisieren
87     pygame.display.update()
88     # Frames per second begrenzen
89     FramePerSec.tick(FPS)

```

Auto im Gegenverkehr: Zusammenstoß melden

Aufgabe: Zusammenstoß melden - Spieler_Klasse_Gegner_Klasse_Zusammenstoss.py

Das Programm bringt ein Fenster mit einem blauen Auto und einem entgegenkommenden roten Auto auf den Bildschirm. *Jedes erfolgreiche Ausweichen des Spielers wird gezählt. Bei einem Zusammenstoß gibt es eine Meldung und das Programm endet.* Starte das Programm. Ändere das Programm und starte es erneut.

Aufgabe	Ergebnis
Steuere das Auto mit den Links- und Rechts-Pfeiltasten der Tastatur. Was passiert bei einem Zusammenstoß der beiden Autos?	
Erzeuge eine weitere Instanz der Gegner-Klasse. Erweitere die Gruppen <code>enemies</code> und <code>all_sprites</code> , sodass zwei rote Autos entgegenkommen.	

```

19 # Farbe definieren
20 WHITE = (255, 255, 255)
21 BLACK = (0, 0, 0)
22 RED = (255, 0, 0)
23 # Schriften definieren
24 font_big = pygame.font.SysFont("Verdana", 40)
25 font_small = pygame.font.SysFont("Verdana", 20)
26 # Text definieren
27 meldung = font_big.render("Zusammenstoß", True, RED)

53 # Gegner-Klasse
54 class Enemy(pygame.sprite.Sprite):
55     def __init__(self):
56         super().__init__()
57         # Bild laden
58         self.image = pygame.image.load("red_car.png")
59         # Rechteck in der Größe des Bildes erzeugen
60         self.rect = self.image.get_rect()
61         # zufällige Anfangsposition
62         self.rect.center = (random.randint(40, SCREEN_WIDTH-40), 0)
63         # score definieren
64         self.score = 0
65     # Enemy bewegen
66     def move(self):
67         self.rect.move_ip(0,10)
68         # Wenn im Fenster unten angekommen
69         if (self.rect.bottom > SCREEN_HEIGHT):
70             # score erhöhen
71             self.score += 1
72             # oben im Fenster beginnen
73             self.rect.top = 0
74             # zufällige Anfangsposition
75             self.rect.center = (random.randint(40, SCREEN_WIDTH - 40), 0)
76 # Instanzen erzeugen
77 P1 = Player()
78 E1 = Enemy()
79 # Gegner-Gruppe erzeugen
80 enemies = pygame.sprite.Group()
81 enemies.add(E1)
82 # Gruppe mit Spieler und Gegner
83 all_sprites = pygame.sprite.Group()
84 all_sprites.add(P1)
85 all_sprites.add(E1)
86 # Funktion beendet das Spiel
87 def end_of_game():
88     # pygame Fenster schließen
89     pygame.quit()
90     # Python Skript beenden
91     sys.exit()

```

```
92 #Game Loop
93 while True:
94     # Ereignisse prüfen
95     for event in pygame.event.get():
96         if event.type == pygame.QUIT:
97             # Spiel-Ende
98             end_of_game()
99     # Grafik-Fenster auswischen
100    screen.fill(WHITE)
101    # score zeichnen
102    my_score = font_small.render(str(E1.score), True, BLACK)
103    screen.blit(my_score, (10,10))
104    # Spieler und Gegner bewegen und zeichnen
105    for entity in all_sprites:
106        # Spieler und Gegner bewegen
107        entity.move()
108        # Blocktransfer: source = entity.image, destination = entity.rect
109        screen.blit(entity.image, entity.rect)
110    # Zusammenstoß entdecken
111    if pygame.sprite.spritecollideany(P1, enemies):
112        # Zusammenstoß melden
113        screen.blit(meldung, (30,250))
114        # Grafik-Fenster aktualisieren
115        pygame.display.update()
116        # Warten
117        time.sleep(5)
118        # Spiel-Ende
119        end_of_game()
120    # Grafik-Fenster aktualisieren
121    pygame.display.update()
122    # Frames per second begrenzen
123    FramePerSec.tick(FPS)
```

Auto im Gegenverkehr: Gegenverkehr wird schneller

Aufgabe: Gegenverkehr wird schneller werden - Spieler_Klasse_Gegner_Klasse_schneller.py

Das Programm bringt ein Fenster mit einem blauen Auto und einem entgegenkommenden roten Auto auf den Bildschirm. Jedes erfolgreiche Ausweichen des Spielers wird gezählt. Bei einem Zusammenstoß gibt es eine Meldung und das Programm endet. *Das rote Auto wird mit jeder Sekunde schneller.* Starte das Programm. Ändere das Programm und starte es erneut.

Aufgabe	Ergebnis
Steuere das Auto mit den Links- und Rechts-Pfeiltasten der Tastatur. Mache 4 Läufe und notiere die Scores.	
Nun soll das Auto jede Sekunde die Geschwindigkeit wechseln. Die Geschwindigkeit ist ein Zufallswert zwischen 5 und 20. Mache 4 Läufe und notiere die Scores.	

```

54 # Gegner-Klasse
55 class Enemy(pygame.sprite.Sprite):
56     def __init__(self):
57         super().__init__()
58         # Bild laden
59         self.image = pygame.image.load("red_car.png")
60         # Rechteck in der Größe des Bildes erzeugen
61         self.rect = self.image.get_rect()
62         # zufällige Anfangsposition
63         self.rect.center = (random.randint(40, SCREEN_WIDTH-40), 0)
64         # score definieren
65         self.score = 0
66         # speed definieren
67         self.speed = 5
68     # Enemy bewegen
69     def move(self):
70         self.rect.move_ip(0, self.speed)
71         # Wenn im Fenster unten angekommen
72         if (self.rect.bottom > SCREEN_HEIGHT):
73             # score erhöhen
74             self.score += 1
75             # oben im Fenster beginnen
76             self.rect.top = 0
77             # zufällige Anfangsposition
78             self.rect.center = (random.randint(40, SCREEN_WIDTH - 40), 0)
79
80 # User-Ereignis INC_SPEED: eindeutige ID definieren
81 INC_SPEED = pygame.USEREVENT + 1
82 # User-Ereignis alle 1000 Millisekunden erscheinen lassen
83 pygame.time.set_timer(INC_SPEED, 1000)
84
85 # Game Loop
86 while True:
87     # Ereignisse prüfen
88     for event in pygame.event.get():
89         if event.type == INC_SPEED:
90             # Gegner schneller machen
91             El.speed += 1
92         if event.type == pygame.QUIT:
93             # Spiel-Ende
94             end_of_game()

```

Auto im Gegenverkehr: Straße und Sound

Aufgabe: Straße im Hintergrund, Sound beim Zusammenstoß – Traffic_Game.py

Das Programm bringt ein Fenster mit einem blauen Auto und einem entgegenkommenden roten Auto auf einer Straße auf den Bildschirm. Jedes erfolgreiche Ausweichen des Spielers wird gezählt. Bei einem Zusammenstoß gibt es eine Meldung und einen Sound und das Programm endet. Das rote Auto wird mit jeder Sekunde schneller. Starte das Programm.

Aufgabe	Ergebnis
Steuere das Auto mit den Links- und Rechts-Pfeiltasten der Tastatur. Wer erreicht den höchsten Score?	

```
26| # Bild für den Hintergrund laden
27| background = pygame.image.load("animated_street.png")

113| # Straße zeichnen
114| screen.blit(background, (0,0))

126| # Sound abspielen
127| pygame.mixer.Sound('crash.wav').play()
```

Ein Dinosaurier überquert die Straße

Aufgabe: Ein Dinosaurier überquert die Straße. Baue das Programm `Traffic_Game.py` um.

Baue das Programm `Traffic_Game.py` Schritt für Schritt um:

1. Kein blaues Auto – Dinosaurier. Kein Zusammenstoß – das Auto bremst rechtzeitig
2. Dinosaurier geht immer mit dem Kopf voran.
3. Auf dem Gehweg ist der Dinosaurier sicher.
4. Jedes erfolgreiche Überqueren der Straße wird gezählt.

Teste das Ergebnis nach jedem Schritt.

Nr.	Aufgabe	Ergebnis
1	Lösche die Zeilen 3 bis 8. Schreibe in Zeile 2: Ein Dinosaurier überquert die Straße Ersetze die Meldung <code>"Zusammenstoß"</code> durch <code>"Achtung!"</code> Ersetze <code>"blue_car.png"</code> durch <code>"dinosaur_LR.png"</code> Ersetze <code>"crash.wav"</code> durch <code>"car_screech.wav"</code>	
2	Spieler-Klasse: Lade 1 Dino-Bild für links → rechts und 1 Dino-Bild für rechts → links: <code>self.image_LR = pygame.image.load("dinosaur_LR.png")</code> <code>self.image_RL = pygame.image.load("dinosaur_RL.png")</code> Definiere das Anfangsbild: <code>self.image = self.image_LR</code> Wechsle die Bilder abhängig von <code>pressed_keys[]</code> : <code>self.image = self.image_RL</code> und <code>self.image = self.image_LR</code> Der Dinosaurier muss schneller laufen, um die Straße zu überqueren: <code>self.rect.move_ip(-10, 0)</code> und <code>self.rect.move_ip(10, 0)</code>	
3	Setze <code>SCREEN_WIDTH = 600</code> Ersetze <code>"animated_street.png"</code> durch <code>"animated_street_w_sidewalk.png"</code> Zu Beginn steht der Dinosaurier auf dem Gehweg. Ersetze in der Spieler-Klasse <code>self.rect.center = (160, 520)</code> durch <code>self.rect.center = (60, 520)</code> Das rote Auto darf nicht auf dem Gehweg fahren. Ersetze in den Methoden <code>__init__()</code> und <code>move()</code> der Gegner-Klasse: <code>self.rect.center = (random.randint(40, SCREEN_WIDTH-40), 0)</code> durch: <code>self.rect.center = (random.randint(150, SCREEN_WIDTH-150), 0)</code>	
4	Das Auto-Spiel berechnet den Score in der Gegner-Klasse. Unser Dinosaurier-Spiel berechnet den Score in der <u>Spieler-Klasse</u> . In der Methode <code>__init__()</code> der Spieler-Klasse werden neue Eigenschaften definiert: <code>self.direction = "right"</code> <code>self.score = 0</code> In der Methode <code>move()</code> der Spieler-Klasse werden zwei neue Bedingungen geprüft: <code># Wenn die Straße überquert wurde</code> <code>if (self.direction == "right") and (self.rect.left > SCREEN_WIDTH-120):</code> <code>self.score += 1</code> <code>self.direction = "left"</code> <code>if (self.direction == "left") and (self.rect.right < 120):</code> <code>self.score += 1</code> <code>self.direction = "right"</code> Die Variable <code>my_score()</code> wird aus <code>P1.score</code> berechnet: <code>my_score = font_small.render(str(P1.score), True, BLACK)</code>	

Ende

Quellen

Quelle	Thema
https://docs.python.org/3/	Python Grundlagen
https://www.python-lernen.de/	Python Grundlagen
Felleisen et al. (2013), Realm of Racket, No Starch Press, San Francisco	Spiel: Computer errät die Zahl
https://www.deinprogramm.de/	Konstruktionsanleitung für Funktionen
https://matplotlib.org/stable/tutorials/pyplot.html	Bibliothek matplotlib
https://docs.python.org/3/library/turtle.html	Bibliothek turtle
https://runestone.academy/ns/books/published/pythonds3/Recursion/ExploringaMaze.html	Irrgarten und Turtle
https://gist.github.com/maxrothman/92eb8470408a047b1f6815a4a444d727	Irrgarten lösen
https://algo.rwth-aachen.de/~algorithmus/algo6.php	Pledge Algorithmus
https://www.pygame.org/docs/index.html	Bibliothek pygame
https://coderslegacy.com/python/python-pygame-tutorial/	Spiel: Traffic Game